

Introduction to Open Publishing

Using Jupyter Book for Open Publishing

Qiusheng Wu¹ ¹University of Tennessee

Abstract

This tutorial introduces open publishing techniques using Jupyter Book and demonstrates how to effectively publish your research.

Whether you write your book’s content in Jupyter Notebooks (.ipynb) or in regular markdown files (.md), you’ll write in the same flavor of markdown called **MyST Markdown**.

1. WHAT IS MYST?

MyST stands for “Markedly Structured Text”. It is a slight variation on a flavor of markdown called “CommonMark” markdown, with small syntax extensions to allow you to write **roles** and **directives** in the Sphinx ecosystem.

2. WHAT ARE ROLES AND DIRECTIVES?

Roles and directives are two of the most powerful tools in Jupyter Book. They are kind of like functions, but written in a markup language. They both serve a similar purpose, but **roles are written in one line**, whereas **directives span many lines**. They both accept different kinds of inputs, and what they do with those inputs depends on the specific role or directive that is being called.

2.a. Using a directive

At its simplest, you can insert a directive into your book’s content like so:

```
```${mydirectivename}  
My directive content
```
```

This will only work if a directive with name `mydirectivename` already exists (which it doesn’t). There are many pre-defined directives associated with Jupyter Book. For example, to insert a note box into your content, you can use the following directive:

```
```${note}  
Here is a note
```
```

This results in:

Note

Here is a note

In your built book.

For more information on writing directives, see the [MyST documentation](#).

2.b. Using a role

Roles are very similar to directives, but they are less-complex and written entirely on one line. You can insert a role into your book's content with this pattern:

Some content {rolename}`and here is my role's content!`

Again, roles will only work if rolename is a valid role's name. For example, the doc role can be used to refer to another page in your book. You can refer directly to another page by its relative path.

For more information on writing roles, see the [MyST documentation](#).

2.c. Adding a figure

You can also add figures to your book.

```
:::{figure} https://assets.gishub.org/images/book-cover.png
:label: gispro-book-cover
:alt: GISPro Book Cover
:align: center
:width: 100%
```

GIS Programming with Python Book Cover.

```
:::
```

Introduction to GIS Programming

Unlock the Power of Python for Geospatial Analysis and Visualization

This book offers a comprehensive, hands-on introduction to the world of geospatial analysis using open-source Python packages. Designed for learners of all levels, this book breaks down the complexities of Geographic Information Systems (GIS) into clear, actionable steps, making it ideal for students, researchers, professionals, and self-learners interested in mastering spatial data programming.

What sets this book apart is its step-by-step, example-driven approach. Beginning with foundational Python programming skills, you'll build your understanding gradually, progressing to advanced techniques in geospatial analysis. The content is designed to be interactive, with real-world datasets and practical exercises that allow you to apply your skills immediately. You'll work through a variety of projects, from basic spatial data manipulation to building interactive dashboards and cloud-based geospatial applications. Whether you're looking to automate GIS workflows, develop geospatial web applications, or deepen your spatial data science skills, this book will guide you through the entire process with clarity and confidence.

What You'll Learn:

- How to set up your development environment using conda, VS Code, Git, Docker, and cloud-computing tools
- Core Python programming skills, from variables and data structures to file I/O and Pandas
- Vector and raster data processing, interactive mapping, 3D visualization, and geoprocessing
- Cloud-based geospatial computing with Google Earth Engine and advanced topics like hyperspectral data and spatial SQL

Dr. Qiusheng Wu is a leading researcher and educator in open geospatial data science. He is the creator of several widely used open-source Python packages, including geemap, leafmap, segment-geospatial, and geoai. Connect with him online:

- <https://x.com/giswqs>
- <https://linkedin.com/in/giswqs>
- <https://youtube.com/@giswqs>
- <https://github.com/giswqs>



Scan the QR code to visit the book website and download code examples from <https://gispro.gishub.org>

Introduction to GIS Programming
Qiusheng Wu

Introduction to GIS Programming

A Practical Python Guide to Open Source Geospatial Tools

Qiusheng Wu

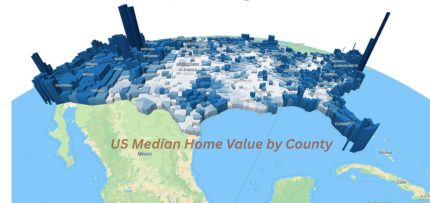


Figure 1: GIS Programming with Python Book Cover.

2.d. Adding a citation

You can also cite references that are stored in a bibtex file. For example, the following syntax: `{cite}`holdgraf_evidence_2014`` will render like this: Holdgraf et al. (2014).

Multiple citations can be used like this: Holdgraf et al. (2016); Holdgraf et al. (2017)

Moreover, you can insert a bibliography into your page with this syntax: The `{bibliography}` directive must be used for all the `{cite}` roles to render properly. For example, if the references for your book are stored in `references.bib`, then the bibliography is inserted with:

```
```{bibliography}
```

Resulting in a rendered bibliography that looks like:

*2.e. Executing code in your markdown files*

If you'd like to include computational content inside these markdown files, you can use MyST Markdown to define cells that will be executed when your book is built. Jupyter Book uses *jupyter* to do this.

First, add Jupyter metadata to the file. For example, to add Jupyter metadata to this markdown page, run this command:

```
jupyter-book myst init markdown.md
```

Once a markdown file has Jupyter metadata in it, you can add the following directive to run the code at build time:

```
```{code-cell}
print("Here is some code to execute")
```
```

When your book is built, the contents of any `{code-cell}` blocks will be executed with your default Jupyter kernel, and their outputs will be displayed in-line with the rest of your content.

For more information about executing computational content with Jupyter Book, see [The MyST-NB documentation](#).

## REFERENCES

- Holdgraf, C. R., Heer, W. de, Pasley, B. N., & Knight, R. T. (2014, ). Evidence for Predictive Coding in Human Auditory Cortex. *International Conference on Cognitive Neuroscience*.
- Holdgraf, C. R., Heer, W. de, Pasley, B. N., Rieger, J. W., Crone, N., Lin, J. J., Knight, R. T., & Theunissen, F. E. (2016). Rapid tuning shifts in human auditory cortex enhance speech intelligibility. *Nature Communications*, 7, 13654. <https://doi.org/10.1038/ncomms13654>
- Holdgraf, C. R., Rieger, J., Micheli, C., Martin, S., Knight, R., & Theunissen, F. (2017). Encoding and decoding models in cognitive electrophysiology. *Frontiers in Systems Neuroscience*, 11. <https://doi.org/10.3389/fnsys.2017.00061>